

PROGRAMMING WITH C++

UNIT 2

FUNCTIONS AND FUNCTIONS TEMPLATES

Outlines

- Function Definitions and Function Prototypes
- Inline Functions
- Function-call stack and Activation Records
- Default argument and object as function argument & returning object
- Friend function
- Static data member and static member function
- Constant member function
- Function Overloading
- Function Templates
- Unary Scope Resolution Operator
- Introduction to Functional-Style Programming
- Recursion

Function Definitions and Function Prototypes

- Functions are fundamental building blocks in C++ programming.
- They allow you to encapsulate a set of instructions to perform a specific task.
- C++ uses both function definitions and function prototypes to enable code organization, reusability, and clarity.

Function Definition

- A function definition provides the actual implementation of a function.
- It includes the function's name, return type, parameter list, and the body of the function, which contains the code that is executed when the function is called.

Syntax:

```
return_type function_name(parameter_list) {  
    // Function body  
    // Statements to be executed  
}
```

- **return_type**: Specifies the type of value the function will return (e.g., int, double, void).
- **function_name**: The name of the function.
- **parameter_list**: A list of parameters (variables) passed to the function (can be empty).
- **Function body**: Contains the statements that define the behavior of the function.

Example

```
#include <iostream>

// Function definition
int add(int a, int b) {
    return a + b; // Adds two integers and returns the result
}

int main() {
    int result = add(5, 3); // Function call
    std::cout << "The sum is: " << result << std::endl; // Output: The sum is: 8
    return 0;
}
```

Output:

The sum is: 8

Function Prototypes

- A function prototype is a declaration of a function, specifying its name, return type, and parameter types, but without the function body.
- Prototypes are typically placed before the `main()` function or in header files to inform the compiler about the function's signature.

Syntax:

```
return_type function_name(parameter_list);
```

- **Function prototype:** This tells the compiler what the function looks like (i.e., its return type and parameters) but does not define the function's actual behavior.

Example

```
#include <iostream>

// Function prototype
int add(int, int);

int main() {
    int result = add(5, 3); // Function call
    std::cout << "The sum is: " << result << std::endl; // Output: The sum is: 8
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

Output:

The sum is: 8

Inline Functions

- Inline function is a function which when invoked requests the compiler to replace the calling statement with its body.
- A keyword **inline** is added before the function name to make it inline.
- It is an optimization technique used by the compilers as it saves time in switching between the functions otherwise.
- Member functions of a class are inline by default even if the keyword **inline** is not used.

Syntax :

```
inline return_type function_name ([argument list])  
{  
    body of function  
}
```


Some situations where inline expansion may not work

- If a function contains a loop. (for, while, do-while)
- If a function contains static variables.
- If a function is recursive.
- If a function return type is other than void, and the
- return statement doesn't exist in function body.
- If a function contains switch or goto statement.

Inline Functions Example

```
#include <iostream>
using namespace std;
inline void print(int x)
{
    cout<<x<< " ";
}
int main() {
    int i, N;
    cout<<"C++ Program to print first N natural numbers"<<endl<<endl;
    cout<<"Enter total number of natural numbers:"<<endl; cin>>N;
    for(i=1;i<=N;i++)
    {
        print(i);
    }
    return 0;
}
```

Output:

```
C++ Program to print first N natural numbers
Enter total number of natural numbers:
10
1 2 3 4 5 6 7 8 9 10
```

Benefits of Inline Functions

- **Reduces Function Call Overhead** – Eliminates the need for stack operations and jumping to a different memory location.
- **Improves Execution Speed** – Faster execution for small, frequently used functions.
- **Enables Compiler Optimizations** – The compiler can better optimize inlined code.
- **Type Safety** – Unlike macros, inline functions are type-checked by the compiler.
- **Better Debugging Support** – Inline functions provide better debugging information compared to macros.
- **Reduces Context Switching Overhead** – Since there is no function call, registers don't need to be saved and restored.
- **Useful for Template Functions** – Helps avoid code duplication while maintaining efficiency.

Function-call stack and Activation Records

- The function-call stack and activation records are integral to how a program handles function calls, manages resources, and maintains control flow.
- **Function-Call Stack:** The **function-call stack** is a runtime data structure used to store information about active function calls in a program.
- **Activation Records (Stack Frames):** An **activation record** is the portion of the call stack allocated for a function call. It contains everything needed to execute and return from the function.

Key Features of the Call Stack

- **Dynamic Nature:**
 - The stack grows when a function is called (pushing a new frame) and shrinks when the function completes (popping the frame).
- **LIFO (Last-In, First-Out):**
 - The most recently called function is the first to finish and return.
- **Storage:**
 - Stores details about each function's execution in units called **activation records** or **stack frames**.
- **Automatic Management:**
 - The system automatically allocates and deallocates memory for functions during execution.

Components of an Activation Record

- **Return Address:** The location in the caller function to which control returns after the function completes.
- **Function Parameters:** Arguments passed to the function by the caller.
- **Local Variables:** Variables declared inside the function.
- **Saved CPU Registers:** Stores the current state of registers to resume execution after the function completes.
- **Dynamic Link:** Points to the base of the caller's stack frame.

How the Call Stack Works

- **Function Call:** When a function is invoked, a new activation record is pushed onto the stack.
- **Function Execution:** The CPU executes the function, using data in the current activation record.
- **Function Return:** After the function completes, its activation record is popped, and control returns to the caller.

Example of Call Stack

```
#include <iostream>
using namespace std;

void secondFunction() {
    int y = 20; // Local variable in secondFunction
    cout << "Inside secondFunction: y = " << y << endl;
}

void firstFunction() {
    int x = 10; // Local variable in firstFunction
    cout << "Inside firstFunction: x = " << x << endl;
    secondFunction(); // Calls secondFunction
}

int main() {
    firstFunction(); // Calls firstFunction
    return 0;
}
```

How the Call Stack Works

main() is called → Activation record for main is pushed.

firstFunction() is called → A new activation record is pushed.

secondFunction() is called → Another activation record is pushed.

secondFunction() completes → Its activation record is popped.

firstFunction() completes → Its activation record is popped.

main() completes → Its activation record is popped.

Example of Activation Record in C++

```
#include <iostream>
using namespace std;

void calculateSum(int a, int b) {
    int sum = a + b; // Local variable
    cout << "Sum: " << sum << endl;
}

void calculateProduct(int x, int y) {
    int product = x * y; // Local variable
    cout << "Product: " << product << endl;
    calculateSum(product, y); // Function call to calculateSum
}

int main() {
    int num1 = 5, num2 = 3; // Local variables in main
    calculateProduct(num1, num2); // Call calculateProduct
    return 0;
}
```

How the Activation Record Works

```
|-----/
| Function: calculateSum() |
| Parameters: a = 15, b = 3 |
| Local Variables: sum = 18 |
| Return Address: calculateProduct() |
|-----/
| Function: calculateProduct() |
| Parameters: x = 5, y = 3 |
| Local Variables: product = 15 |
| Return Address: main() |
|-----/
| Function: main() |
| Local Variables: num1 = 5, num2 = 3 |
| Return Address: Program End |
|-----/
```

Default argument in Function

- In C++, default arguments allow you to provide default values for function parameters.
- If the caller does not pass an argument for a parameter, the default value is used instead.
- This feature helps in making functions more flexible and reduces the need for function overloading.

Syntax:

```
return_type function_name(type param1 = default_value1, type param2 = default_value2, ...);
```

Default argument in Function

```
#include <iostream>
using namespace std;

// Function with a default argument
void printMessage(string message = "Hello, World!") {
    cout << message << endl;
}

int main() {
    printMessage();           // Uses default argument: "Hello, World!"
    printMessage("Custom msg"); // Uses provided argument: "Custom msg"
    return 0;
}
```

Output:

```
Hello, World!
Custom msg
```

Default argument in Function

```
#include <iostream>
using namespace std;

void display(int a, int b = 10, int c = 20) {
    cout << "a = " << a << ", b = " << b << ", c = " << c << endl;
}

int main() {
    display(1);    // Uses default values for b and c
    display(1, 5); // Uses default value for c
    display(1, 5, 15); // No default values used
    return 0;
}
```

Output:

```
a = 1, b = 10, c = 20
a = 1, b = 5, c = 20
a = 1, b = 5, c = 15
```

object as function argument

In C++, you can pass objects as arguments to functions. Objects can be passed in three ways:

- **By Value:** The object is passed by creating a copy.
- **By Reference:** The function works with the original object, so modifications inside the function will affect the original.
- **By Pointer:** The function works with the pointer to the object, allowing modifications to the original object.

Passing Objects by Value

- When you pass an object by value, a copy of the object is created and passed to the function. This means that the original object is not modified by the function. This is usually less efficient if the object is large because it involves a copy.

```
#include <iostream>
using namespace std;

class MyClass {
public:
    int value;
    MyClass(int v) {
        value = v;
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};
```

```
void processObject(MyClass obj) {
    obj.value = 20; // Modifies the copy, not the original object
    obj.display(); // Displays modified value (only inside function)
}

int main() {
    MyClass obj(10); // Create object with value 10
    processObject(obj); // Pass by value
    obj.display(); // Original object is not modified
    return 0;
}
```

Output:

```
Value: 20
Value: 10
```

Passing Objects by Reference

- When an object is passed by reference, no copy of the object is made. The function works directly with the original object, and any changes made inside the function will affect the original object. This is generally more efficient, especially for large objects.
- You can pass by reference using the & symbol.

```
#include <iostream>
using namespace std;

class MyClass {
public:
    int value;
    MyClass(int v) {
        value = v;
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};
```

```
void processObject(MyClass& obj) {
    obj.value = 20; // Modifies the original object
    obj.display(); // Displays modified value
}

int main() {
    MyClass obj(10); // Create object with value 10
    processObject(obj); // Pass by reference
    obj.display(); // The original object is modified
    return 0;
}
```

Output:

```
Value: 20
Value: 20
```

Passing Objects by Pointer

- When you pass an object by pointer, you're passing the address of the object, which allows the function to access and modify the original object. You can also check for nullptr if needed.
- You can pass by pointer using the * symbol.

```
#include <iostream>
using namespace std;

class MyClass {
public:
    int value;
    MyClass(int v) {
        value = v;
    }

    void display() const {
        cout << "Value: " << value << endl;
    }

    void setValue(int v) {
        value = v;
    }
};
```

```
void processObject(MyClass* obj) {
    if (obj != nullptr) {
        obj->setValue(20); // Modifies the original object
        obj->display(); // Displays modified value
    }
}

int main() {
    MyClass obj(10); // Create object with value 10
    processObject(&obj); // Pass by pointer (passing address)
    obj.display(); // The original object is modified
    return 0;
}
```

Output:

```
Value: 20
Value: 20
```


Function returning object

- In C++, a function can return an object. This is particularly useful when implementing functionality that generates or transforms objects.

```
#include <iostream>

class Rectangle {
private:
    double length, width;

public:
    Rectangle(double l, double w) : length(l),
    width(w) {}

    double area() const {
        return length * width;
    }
};
```

```
// Function returning a new Rectangle object
Rectangle createSquare(double side) {
    return Rectangle(side, side); // Creating and returning a
    square
}

int main() {
    Rectangle square = createSquare(4.0); // Capture the
    returned object
    std::cout << "Square area: " << square.area() << std::endl;

    return 0;
}
```

Output:

Square area: 16

Friend function

- In C++, a **friend function** is a function that is not a member of a class but is allowed to access the private and protected members of the class.
- A friend function can be a normal function, a method of another class, or even a global function.
- The key idea is that friendship is granted by the class, not by the scope of the function.

Key points about Friend Functions

- **Access to private/protected members:** A friend function has access to the private and protected members of the class, even though it is not a member itself.
- **Declaration:** The function is declared inside the class using the keyword friend.
- **Non-member function:** A friend function is not part of the class. It can be a global function or a function in another class.
- **No special return type or argument requirements:** A friend function looks just like a regular function but is given special access privileges.

Example of Friend Function

```
#include <iostream>
using namespace std;

class Box {
private:
    double length; // Private member
public:
    Box(double l) : length(l) {} // Constructor to initialize length

    // Declare friend function
    friend double getLength(const Box& b);
};
```

```
// Friend function definition
double getLength(const Box& b) {
    return b.length; // Access private member
}

int main() {
    Box b(10.5);

    // Use the friend function to access private member
    cout << "Length of the box: " << getLength(b) << endl;

    return 0;
}
```

Output:

Length of the box: 10.5

Static data member and static member function

- In C++, static data members and static member functions are associated with the class itself rather than any specific object of the class.
- This means that they are shared across all instances (objects) of the class.
- These members have specific behavior and usage rules, which are different from non-static members.

Advantages of Static Data Members and Static Member Functions:

- **Memory Efficiency:** Since there is only one copy of a static data member for the entire class, it saves memory when the data is shared among all instances.
- **Shared Data:** Static data members can be used to store values that are shared across all objects, like a global counter or configuration setting.
- **No Need for Object Creation:** Static member functions can be called without creating an object, which is useful for utility functions that don't depend on instance-specific data.

Static Data Member

- A static data member is a variable that is shared by all objects of the class.
- There is only one copy of the static data member, and it is common to all objects of the class.
- Static data members are useful for storing class-wide information, such as counters or constants.

Key Characteristics:

- **Shared across objects:** Static data members are not tied to any specific object. All objects of the class share the same copy of the static data member.
- **Accessed using the class name:** Static data members are typically accessed using the class name rather than through an object instance.
- **Defined outside the class:** While a static data member is declared inside the class, it must be defined outside the class in a source file (typically a .cpp file).

Example of static member

```
#include <iostream>

class MyClass {
public:
    static int count; // Static data member declaration

    MyClass() {
        count++;
    }

    static void showCount() {
        std::cout << "Count: " << count << std::endl;
    }
};
```

```
// Static data member definition (outside the class)
int MyClass::count = 0;

int main() {
    MyClass obj1;
    obj1.showCount(); // Output: Count: 1
    MyClass obj2;
    obj2.showCount(); // Output: Count: 2
    return 0;
}
```

Output:

```
Count: 1
Count: 2
```

Static Member Function

- A static member function is a function that belongs to the class rather than any object of the class.
- Since static functions do not have a this pointer, they can only access static data members or call other static member functions.
- Static member functions cannot access non-static data members or non-static member functions directly.

Key Characteristics:

- **Does not require an object:** Static member functions can be called on the class itself, not needing an instance of the class.
- **Can only access static members:** Since they don't have a this pointer, static member functions can only access static data members or other static member functions.
- **Can be called without creating an object:** Static member functions are typically called using the class name.

Example of static member function

```
class MyClass {  
public:  
    static int count; // Static data member  
    static void incrementCount() {  
        count++; // Static member function can modify static data  
    }  
  
    static void showCount() {  
        std::cout << "Count: " << count << std::endl;  
    }  
};
```

```
// Static data member definition (outside the class)  
int MyClass::count = 0;  
  
int main() {  
    MyClass::incrementCount(); // Calling static function without an object  
    MyClass::showCount();     // Calling static function without an object  
  
    MyClass obj1;  
    obj1.showCount();         // Can also call static function using an object  
    return 0;  
}
```

Output:

```
Count: 1  
Count: 1
```

constant member function

- In C++, a constant member function (also known as a const member function) is a member function that guarantees not to modify the state of the object it is called on.
- It ensures that no non-const data members or non-const member functions can be modified within that function.
- Constant member functions are used when you want to provide functionality that reads but does not modify the object.

Syntax:

```
returnType functionName() const;
```

Key Points

- **Read-Only Access:** Constant member functions can only read member variables, not modify them.
- **Compile-Time Enforcement:** If you attempt to modify any member variables in a const function, the compiler will generate an error.
- **Calling on Const Objects:** A constant member function can be called on both constant and non-constant objects. However, a non-constant member function cannot be called on a constant object.

Example of constant member function

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int length, width;

public:
    // Constructor to initialize the rectangle
    Rectangle(int l, int w) : length(l), width(w) {}

    // Constant member function: It only reads data, doesn't modify it
    int getArea() const {
        return length * width; // allowed, because this function doesn't modify the object
    }

    // Non-constant member function: It can modify data
    void setLength(int l) {
        length = l;
    }
}
```

```
void setWidth(int w) {
    width = w;
}

int main() {
    Rectangle rect(10, 5); // Create an object of Rectangle

    // Calling constant member function to get the area
    cout << "Area: " << rect.getArea() << endl; // Output: Area: 50

    // Calling non-constant functions to modify the object
    rect.setLength(20);
    rect.setWidth(10);

    // Calling constant member function again after modification
    cout << "New Area: " << rect.getArea() << endl; // Output: New Area: 200

    return 0;
}
```

Output:

```
Area: 50
New Area: 200
```

Function Overloading

- Function overloading in C++ is a feature that allows multiple functions to have the same name but differ in the number or types of their parameters.
- The compiler differentiates between the overloaded functions based on their signature, which includes the number of parameters, the type of parameters, and the order of parameters.

Key Points about Function Overloading

- **Same function name:** You can have multiple functions with the same name.
- **Different parameter list:** The functions must differ in the number or types of parameters.
- **Return type doesn't matter:** Overloading cannot be based on different return types alone.

Example of function overloading

```
#include <iostream>
using namespace std;

class Print {
public:
    // Overloaded function to print an integer
    void display(int x) {
        cout << "Integer: " << x << endl;
    }

    // Overloaded function to print a float
    void display(float x) {
        cout << "Float: " << x << endl;
    }
}
```

```
// Overloaded function to print a string
void display(string x) {
    cout << "String: " << x << endl;
}

int main() {
    Print obj;

    obj.display(10);    // Calls display(int)
    obj.display(3.14f); // Calls display(float)
    obj.display("Hello!"); // Calls display(string)

    return 0;
}
```

Output:

```
Integer: 10
Float: 3.14
String: Hello!
```

Function Templates

- A function template is a blueprint or a generic function that can work with any data type.
- Instead of writing multiple functions for different data types, you can write a single template function that works for all types, using the template keyword.
- This is especially useful when you want the same functionality for different data types (like int, float, double, etc.).

Syntax:

```
template <typename T>  
return_type function_name(T parameter1, T parameter2) {  
    // function body  
}
```

Key Points

- **Generic Function:** A function template allows you to define a single function that can operate on different data types.
- **template Keyword:** The template keyword is used to define a function template.
- **Type Parameter:** A placeholder (usually denoted as T, but you can choose any valid name) is used to represent the data type.
- **Automatic Type Deduction:** The compiler can automatically deduce the type of the arguments when you call the function.

Example of function Templates

```
#include <iostream>
using namespace std;

// Template for a generic function to find the maximum of two values
template <typename T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    // Calling template function with different types
    cout << "Max of 10 and 20: " << findMax(10, 20) << endl;    // int
    cout << "Max of 10.5 and 7.3: " << findMax(10.5, 7.3) << endl; // double
    cout << "Max of 'A' and 'Z': " << findMax('A', 'Z') << endl; // char

    return 0;
}
```

Output:

```
Max of 10 and 20: 20
Max of 10.5 and 7.3: 10.5
Max of 'A' and 'Z': Z
```

Unary Scope Resolution Operator

- The unary scope resolution operator (::) in C++ is used to access global variables, functions, or classes when they are hidden by local variables or classes.
- It helps in referring to global entities explicitly, even if there is a local variable or class with the same name.

Key Points:

- The scope resolution operator :: allows you to access global scope when there is a name conflict between local and global identifiers.
- It can also be used to define methods outside the class in the global scope or to access static members of a class.

Usage:

- **Accessing Global Variables or Functions:** If there is a local variable with the same name as a global variable, you can use the scope resolution operator :: to access the global variable.
- **Accessing Class Members:** You can use :: to define a function outside the class or to access static class members.

Accessing Global Variable from Inside a Function

```
#include <iostream>
using namespace std;

int x = 10; // Global variable

void printX() {
    int x = 20; // Local variable
    cout << "Local x: " << x << endl;
    cout << "Global x: " << ::x << endl; // Access global variable using scope resolution operator
}

int main() {
    printX(); // Call the function
    return 0;
}
```

Output:

```
Local x: 20
Global x: 10
```

Defining a Class Method Outside the Class

```
#include <iostream>
using namespace std;

class MyClass {
private:
    int x;

public:
    MyClass(int val) : x(val) {}

    // Declaration of the member function
    void printX();
};
```

```
// Definition of the member function using scope resolution operator
void MyClass::printX() {
    cout << "Value of x: " << x << endl;
}

int main() {
    MyClass obj(100);
    obj.printX(); // Call the printX function
    return 0;
}
```

Output:

```
Value of x: 100
```

Accessing Static Members of a Class

```
#include <iostream>
using namespace std;

class MyClass {
public:
    static int count; // Static variable

    static void displayCount() { // Static function
        cout << "Count: " << count << endl;
    }
};
```

```
// Definition of static member outside the class
int MyClass::count = 0;

int main() {
    MyClass::count = 10; // Access static variable using scope
    resolution
    MyClass::displayCount(); // Access static function using scope
    resolution

    return 0;
}
```

Output:

Count: 10

Introduction to Functional-Style Programming

- Functional programming in C++ is a style that emphasizes immutability, higher-order functions, and treating functions as first-class citizens.
- Although C++ is traditionally an object-oriented language, modern versions (especially C++11 and beyond) provide features that make it possible to write C++ code in a functional programming style.

Key Concepts of Functional Programming

- **First-Class Functions**: Functions can be passed as arguments, returned from other functions, and assigned to variables.
- **Immutability**: Data is not modified after it is created. This makes code easier to reason about and avoids side effects.
- **Pure Functions**: Functions that have no side effects and return the same output for the same input.
- **Higher-Order Functions**: Functions that take other functions as parameters or return them.
- **Laziness/Deferred Execution**: Deferring the evaluation of an expression until its value is needed.

First-Class Functions example

```
#include <iostream>
using namespace std;

int add(int x, int y) {
    return x + y;
}

int apply_function(int (*func)(int, int), int a, int b) {
    return func(a, b);
}

int main() {
    int result = apply_function(add, 2, 3);
    cout << result << endl; // Output: 5
    return 0;
}
```

Recursion

Recursion is a powerful programming technique where a function calls itself to solve smaller instances of the same problem. A recursive function typically has two main components:

- **Base Case:** A condition that stops the recursion (prevents infinite recursion).
- **Recursive Case:** The part where the function calls itself with modified parameters, gradually

Advantages of Recursion:

- **Simplicity:** Recursive solutions often lead to simpler and more elegant code, especially for problems like tree traversal, searching, and divide-and-conquer algorithms (like quicksort).
- **Decomposing Complex Problems:** Recursion helps in breaking down problems into smaller subproblems, often making the code easier to understand.

Example of Recursion

```
#include <iostream>
using namespace std;

// Recursive function to calculate factorial
int factorial(int n) {
    if (n == 0) {
        return 1; // Base case
    } else {
        return n * factorial(n - 1); // Recursive case
    }
}

int main() {
    int number = 5;
    cout << "Factorial of " << number << " is " << factorial(number) <<
endl;
    return 0;
}
```

Output:

Factorial of 5 is 120